

Structure-Based Prediction of NES Motifs using GNNs

Zvi Marmor, Amit Yosef, Shaked Hartal, Hadar Dayan
Hebrew University of Jerusalem

Hackathon Presentation – 3D Data Processing in Structural Biology

Project Goal

Objective: Predict NES (Nuclear Export Signals) in proteins using 3D structural data and Graph Neural Networks.

- ▶ Utilize AlphaFold-predicted PDB files.
- ▶ Construct graphs where nodes = residues, edges = spatial proximity.
- ▶ Use GNNs to classify NES presence.

Data and Preprocessing

- ▶ 675 protein structures (AlphaFold PDBs).
- ▶ Labels: NES-positive / NES-negative.
- ▶ Each residue: C-alpha atom used as graph node.
- ▶ Edges between residues j 8Å apart.

Graph Construction Pipeline

1. Filter to chain B (NES motif) + surrounding residues ($\pm 15\text{\AA}$).
2. Encode residues: One-hot + NES-chain binary.
3. Build PyTorch Geometric graph (x , edge_index, edge_attr, pos).
4. Optional: export filtered PDB and visualize.

Visualizations

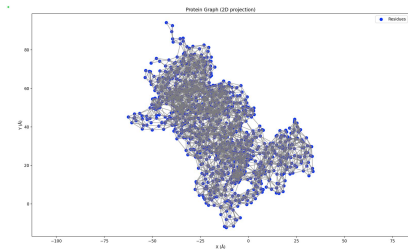


Figure 1: Full dense protein graph.

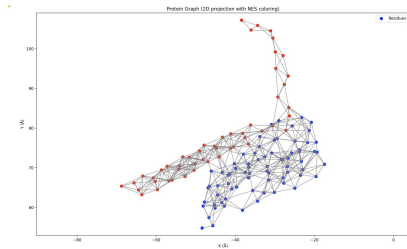


Figure 2: NES chain (red) + nearby residues (blue).

Model Selection: EGNN vs. GVP-GNN

Feature	GVP-GNN	EGNN
Handles vectors?	Yes	No
Equivariance	SE(3)	Rotation/Translation/Reflection
Complexity	High	Moderate
Data Efficiency	Low	High
Best for CA atoms?	No	Yes

Why EGNN?

- ▶ Our graphs only use distances (not full vectors).
- ▶ about 600 examples – EGNN less prone to overfitting.
- ▶ Simpler, faster for experimentation during hackathon.

Model Architecture (EGNN)

- ▶ Node features: amino acid type + NES-chain binary.
- ▶ Edge features: Euclidean distance.
- ▶ Node/pos updated jointly in equivariant fashion.
- ▶ Output: classification (NES present or not).

Baseline Model: GCN (Graph Convolutional Network)

What is GCN?

- ▶ Neural network designed for graph-structured data (like proteins).
- ▶ Acts like CNNs but for graphs — aggregates neighbor features.

How It Works

1. Each node (residue) has a one-hot vector for amino acid type.
2. Edges connect $C\alpha$ atoms within a certain distance.
3. Feature updates via weighted aggregation from neighbors.
4. Global pooling used for whole-protein classification.

Baseline Model: GCN (Graph Convolutional Network)

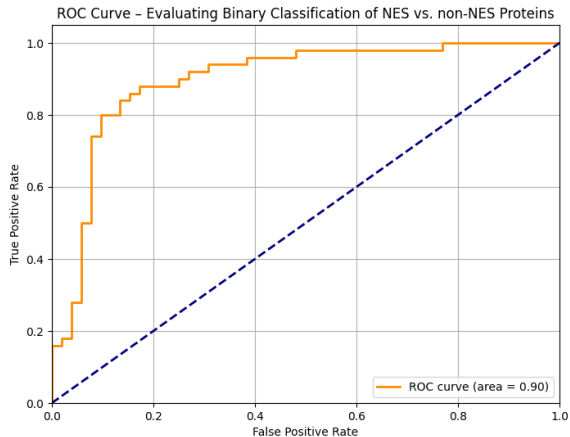
Pros

- ▶ Simple and efficient.
- ▶ Validates preprocessing pipeline.
- ▶ Good for early experimentation.

Cons

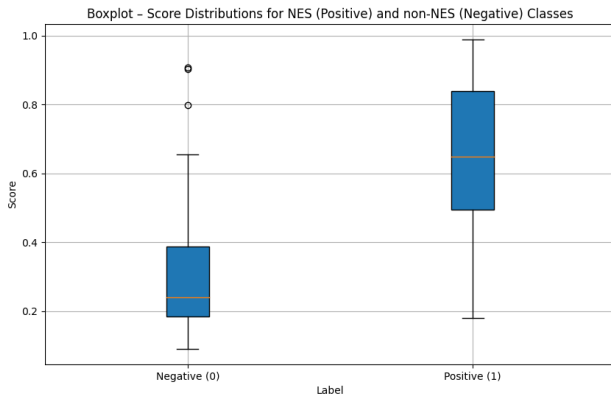
- ▶ Ignores 3D geometry (no distance or positional data).
- ▶ Less expressive than EGNN or GVP-GNN.

Model Performance: ROC Curve



ROC curve visualizing the trade-off between true positive rate and false positive rate. $AUC = 0.90$ indicates excellent discrimination.

Model Performance: Score Distributions



Boxplot showing predicted probability distributions for NES-positive and NES-negative samples.

Evaluation Metrics Used

To evaluate NES classification beyond accuracy, we used:

Metric	Description	Importance
Accuracy	Correct predictions overall	General performance
Precision	$TP / (TP + FP)$	Avoid false positives
Recall	$TP / (TP + FN)$	Catch NES cases
F1 Score	Harmonic mean of P/R	Balances precision/recall
ROC AUC	Area under ROC curve	Discrimination power
Confusion Matrix	TP, FP, TN, FN breakdown	Error breakdown
PR Curve	Precision vs. recall	Better for imbalanced data

Optimization and Diagnostics

Dropout Regularization

- ▶ Dropout = 0.3 $\rightarrow$ AUC = 0.881, Accuracy = 80.4
- ▶ Dropout = 0.5 $\rightarrow$ Similar performance, no major gain

Optimizer Comparison

- ▶ **Adam:** AUC = 0.88, stable training
- ▶ AdamW: Similar to Adam, better weight decay
- ▶ **SGD:** AUC = 0.65, unstable and low accuracy

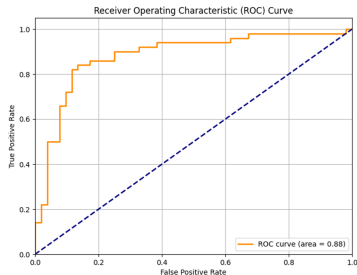
Takeaway: Adaptive optimizers + modest dropout yielded best results.

Training and Evaluation Insights

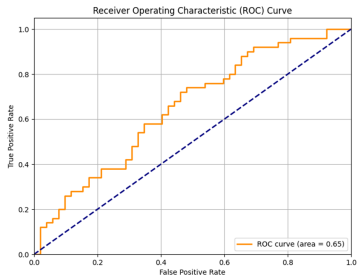
- ▶ **Initial training loss:** 0.693 (balanced binary)
- ▶ **Validation loss** increased around epoch 23 → early overfitting
- ▶ **Final performance (Adam):**
 - ▶ $\text{AUC} = 0.88$
 - ▶ $\text{Accuracy} = 82.4$
- ▶ **Benchmark AUC:** 0.9148 — our model showed close performance.

Next steps: improve generalization via regularization, embeddings, and architectural tuning.

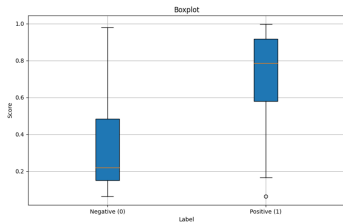
Optimizer Comparison: Visual Results



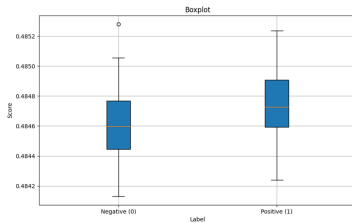
Adam Optimizer – ROC AUC = 0.88



SGD Optimizer – ROC AUC = 0.65



Prediction scores – Clear separation



Prediction scores – Poor separation

Future Work

- ▶ Integrate ESM embeddings + pLDDT.
- ▶ Attention visualization on NES pockets.
- ▶ Try SE(3)-Transformer on same graph structure.

Thank You!

Questions? Feedback?

Team: Tzvi Marmor, Amit Joseph, Shaked Hartal, Hadar Dayan